Machine Learning - 2026

Kernel PCA

Dr. Suresh Manandhar

Chair Professor of AI

Madan Bhandari University of Science and Technology

Dot products again

- **Dot products between vectors:**

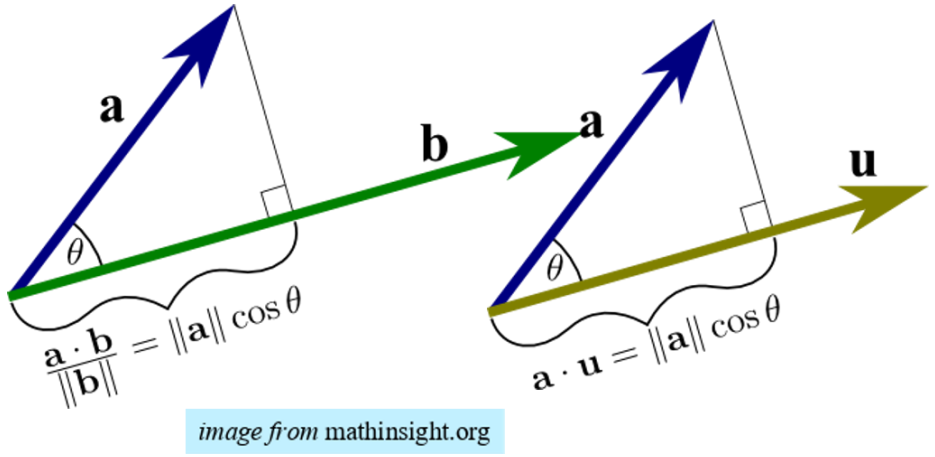


image from mathinsight.org

- corresponds to the cosine of the angle between the vectors
- $a.b = ||a||\ ||b|| \cos\theta$
- Dot products (the normalized cosine) measure the ***similarity*** of two data points in a vector space
- But ***similarity*** is an ***abstract concept***

Key ideas

- The question we want to ask is?
  - What if we only have similarity information between data points?
  - i.e. do not have representation in feature space (i.e. $\Phi(x)$ space)
  - In other words, we do have the data, but we do not know what the good features are, hence we have **not** done the feature transformation $\Phi(x)$
  - For example, if $x = \langle x_1, x_2 \rangle$ then $\Phi(x)$ could be
  - $\Phi(x) = \langle 1, x_1, x_2, x_1^2, x_2^2, x_1 x_2 \rangle$
- So, we have some way to computing dot products in the unknown space given by some transformation :
  - $\Phi(x) \cdot \Phi(y)$
- We actually do not know $\Phi(x)$, we just know what the dot products are in the transformed space.

Key ideas

- If we are in such a scenario, then:
  - will there exist a ***vector space*** corresponding to the one mapped by the ***'fictitious'*** $\Phi(x)$?
  - or, more precisely, a ***Hilbert space***
- Hilbert spaces generalise vectors spaces to spaces that have dot products defined (and behaving like dot products).
- **Kernel Trick**: We define a kernel K to be implicitly computing the dot products in the new feature space:
  - $K(x,y) = \Phi(x) \cdot \Phi(y)$
- So, K will be a function that we can program which we will then supply as a subroutine to a kernelized machine learning algorithm.

Mercer's theorem

- **If :**
  - K is continuous real valued function
    - So, K does not have discontinuities
  - K is symmetric – i.e. $K(x,y) = K(y,x)$ for any vectors x,y
  - K is positive semi-definite (Mercer condition):
  - K is positive semi-definite if for any vectors $\{x_1, \ldots, x_n\}$ and constants $\{c_1, \ldots, c_n\}$:

$$\sum_i \sum_j c_i c_j K\left(x_{i,} x_j\right) ge0$$

  - The above condition is easily met if $K(x,y) \geq 0$ for any vectors x,y
- **then** :
  - there is a Hilbert space **H** and a mapping $\Phi(x)$ in **H** such that:
    - $K(x,y) = \Phi(x) \cdot \Phi(y)$

Kernel Matrix

- Given data set $\boldsymbol{X}$ with each row of $\boldsymbol{X}$ containing data vector $\boldsymbol{x_i}$
- The kernel matrix (also known as the ***gram matrix***) is defined by:
  - $X\,X^T$
- So, if $\boldsymbol{X}$ contains data points $\{\boldsymbol{x_1}, \ldots, \boldsymbol{x_N}\}$ in its rows
- Then $\boldsymbol{X^T}$ will contain data points $\{\boldsymbol{x_1}, \ldots, \boldsymbol{x_N}\}$ in its column
- So, the kernel matrix $\boldsymbol{X}\,\boldsymbol{X^T}$ will be given by:

$$XX^T = \begin{bmatrix} x_1.x_1 & \ldots & x_1.x_N \\ x_2.x_1 & \ldots & x_2.x_N \\ \ldots & \ldots & \ldots \\ x_N.x_1 & \ldots & x_N.x_N \end{bmatrix}$$

Kernel Matrix

- If we map/translate each data point $\boldsymbol{x}$ to kernel/Hilbert space by $\boldsymbol{\Phi(x)}$
- then the kernel matrix becomes:

$$K = \begin{bmatrix} \varphi(x_1).\varphi(x_1) & \dots & \varphi(x_1).\varphi(x_N) \\ \varphi(x_2).\varphi(x_1) & \dots & \varphi(x_2).\varphi(x_N) \\ \dots & \dots & \dots \\ \varphi(x_N).\varphi(x_1) & \dots & \varphi(x_N).\varphi(x_N) \end{bmatrix}$$

- Using the ***kernel trick***, this becomes:

$$K = \begin{bmatrix} k(x_1,x_1) & \dots & k(x_1,x_N) \\ k(x_2,x_1) & \dots & k(x_2,x_N) \\ \dots & \dots & \dots \\ k(x_1) & \dots & k(x_N) \end{bmatrix}$$

- The term kernel matrix is used somewhat ambiguously to mean either $\boldsymbol{K}$ above or $\boldsymbol{X}\,\boldsymbol{X}^T$
- When a kernel mapping/function is involved then we mean $\boldsymbol{K}$
- To avoid any confusion, we explicitly write $\boldsymbol{K}$ or $\boldsymbol{X}\,\boldsymbol{X}^T$

Some example kernels

- ***Polynomial Kernel:*** for real c, d

$$k(x,y) = (x.y + c)^d$$

- For d = 2:

$$k(x,y) = \left(c + \sum_i x_i y_i\right)^2 = \sum_i \sum_j x_i y_i x_j y_j + 2c \sum_i x_i y_i + c^2$$

$$= \sum_i \sum_j x_i x_j y_i y_j + \sum_i \left(\sqrt{2c}x_i\right)\left(\sqrt{2c}y_i\right) + c^2$$

$$= \left\langle \ldots, x_i x_j, \ldots, \ldots, \sqrt{2c}x_i, \ldots, c \right\rangle \left\langle \ldots, y_i y_j, \ldots, \ldots, \sqrt{2c}y_i, \ldots, c \right\rangle$$

- Thus, the above polynomial feature expansion is equivalent to a single call to the kernel.
- This derivation proves (by explicit construction of the feature/Hilbert space) that the polynomial kernel is indeed a kernel.

Radial basis (RBF) or Gaussian Kernel

- For :

$$k(x,y)=\exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

$$1 = -\frac{1}{2\sigma^2}$$

$$k(x,y)=\exp\left(\|x-y\|^2\right)=\exp\left(\|x\|^2-2x^Ty+\|y\|^2\right)$$

- using Taylor series expansion of e^z with $z=x^Ty$:

$$e^z=1+z+\cdots+\frac{1}{i!}z^k+\cdots=\sum_{i=0}^{\infty}\frac{z^i}{i!}=\sum_{i=0}^{\infty}\frac{(x.y)^i}{i!}$$

$$\exp\left(\|x\|^2-2x^Ty+\|y\|^2\right)=\sum_{i=0}^{\infty}\frac{(x.y)^i}{i!}\exp\left(\|x\|^2\right)\exp\left(\|y\|^2\right)$$

- From this it can be seen that the RBF kernel (is a kernel and) has a feature representation that involves an **infinite** expansion of polynomial terms.
- Each $(x,y)^i$ term is an order **i** polynomial kernel as defined earlier.

Kernel PCA

- PCA solves the optimisation problem

$$W^* = \arg\max_{W^T W = I} W^T X^T X W$$

- If we let $W = X^T a$ we get:

$$\begin{aligned} W^T X^T X W &= (X^T a)^T X^T X (X^T a) \\ &= a^T X X^T X X^T a \\ &= a^T K K a \end{aligned}$$

- **Definition**: Kernel PCA problem is to find solution to the optimisation problem:

$$W^* = \arg\max_{W^T W = I} W^T K K W$$

- **Solving Kernel PCA** amounts to finding eigen decomposition for KK

- We note that, if the eigen decomposition of $\boldsymbol{K}$ is given by:

$$\boldsymbol{K} = \boldsymbol{W}\boldsymbol{\Lambda}\boldsymbol{W}^T$$

then:

$$\boldsymbol{K}\boldsymbol{K} = \boldsymbol{W}\boldsymbol{\Lambda}\boldsymbol{W}^T\boldsymbol{W}\boldsymbol{\Lambda}\boldsymbol{W}^T = \boldsymbol{W}\boldsymbol{\Lambda}\boldsymbol{I}\boldsymbol{\Lambda}\boldsymbol{W}^T$$

$$= \boldsymbol{W}\boldsymbol{\Lambda}\boldsymbol{\Lambda}\boldsymbol{W}^T = \boldsymbol{W}\boldsymbol{\Lambda}^2\boldsymbol{W}^T$$

- Hence, the *eigen vectors* for $\mathbf{K}^2$ are same as for **K**
- and, the *eigen values* for $\mathbf{K}^2$ are given by square of the eigen values of **K**

Centering the Kernel matrix

- Since eigen decomposition requires that the input matrix is zero mean we first need to center the kernel matrix

The following centering is actually ***incorrect*** since it only centers the columns and does not result in a symmetric matrix.

$$\boldsymbol{K}' = \begin{bmatrix} k_{11}-\mu_1 & k_{21}-\mu_2 & k_{31}-\mu_3 & k_{41}-\mu_4 \\ k_{12}-\mu_1 & k_{22}-\mu_2 & k_{32}-\mu_3 & k_{42}-\mu_4 \\ k_{13}-\mu_1 & k_{23}-\mu_2 & k_{33}-\mu_3 & k_{43}-\mu_4 \\ k_{14}-\mu_1 & k_{24}-\mu_2 & k_{34}-\mu_3 & k_{44}-\mu_4 \end{bmatrix}$$

where $\boldsymbol{\mu_i}$ is the mean for column $\boldsymbol{i}$

Centering the Kernel matrix

- Since $K_{ij} = \Phi_i \cdot \Phi_j = \Phi_i \Phi_j^T$
- We need to replace each transformation Φ_i with $\Phi_i - \bar{\Phi}$ where, $\bar{\Phi}$ is the mean.
- Thus, the centered kernel can be written as:

$$K_{ij}^c = \left(\Phi_i - \frac{1}{N}\sum_l \Phi_l\right)\left(\Phi_j - \frac{1}{N}\sum_m \Phi_m\right)^T = \left(\Phi_i - \frac{1}{N}\sum_l \Phi_l\right)\left(\Phi_j^T - \frac{1}{N}\sum_m \Phi_m^T\right)$$

$$= \Phi_i\Phi_j^T - \Phi_i\frac{1}{N}\sum_m \Phi_m^T - \frac{1}{N}\left(\sum_l \Phi_l\right)\Phi_j^T + \left(\frac{1}{N}\sum_l \Phi_l\right)\left(\frac{1}{N}\sum_m \Phi_m^T\right)$$

Centering the Kernel matrix

$$K^c_{ij} = \phi_i \phi_j^T - \phi_i \frac{1}{N} \sum_m \phi_m^T - \phi_j^T \frac{1}{N} \sum_l \phi_l + \left(\frac{1}{N} \sum_l \phi_l \right) \left(\frac{1}{N} \sum_m \phi_m^T \right)$$

- Now, we can further simplify using K :

$$\phi_i \sum_m \phi_m^T = \sum_m \phi_i \phi_m^T = \begin{bmatrix} K_{i1} \dots K_{iN} \end{bmatrix} \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} = K_{i*} \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$$

$$\left(\sum_l \phi_l \right) \phi_j^T = \sum_l \phi_l \phi_j^T = \begin{bmatrix} 1 \dots 1 \end{bmatrix} \begin{bmatrix} K_{j1} \\ \vdots \\ K_{jN} \end{bmatrix} = \begin{bmatrix} 1 \dots 1 \end{bmatrix} K_{*j}$$

$$\left(\sum_l \phi_l \right) \left(\sum_m \phi_m^T \right) = \sum_{l,m} \phi_l \phi_m^T = \begin{bmatrix} 1 \dots 1 \end{bmatrix} K \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$$

- Substituting, we get:

$$K^c_{ij} = K_{ij} - \frac{1}{N} K_{i*} \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} - \frac{1}{N} \begin{bmatrix} 1 \dots 1 \end{bmatrix} K_{*j} + \frac{1}{N^2} \begin{bmatrix} 1 \dots 1 \end{bmatrix} K \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$$

- Or for the full matrix:

$$K^c = K - \frac{1}{N} K\,1 - \frac{1}{N} 1\,K + \frac{1}{N^2} 1\,K\,1$$

(verify this out on paper)

Kernel PCA procedure

- **Purpose**: perform non linear dimensionality reduction in kernel space
- **Method**:
  - Using the kernel function, compute the kernel matrix ***K***
  - Center the kernel matrix ***K*** using:

$$K^c = K - \frac{1}{N} K\,1 - \frac{1}{N} 1\,K + \frac{1}{N^2} 1\,K\,1$$

  - Or perform eigen decomposition on ***K***
    - Sort the eigen values in decreasing order
    - Remove negative eigen values (and eigen vectors)
    - Choose the top **k** eigen vectors as the new bases
  - To choose **k**, one can use the following method:
    - Compute running total of all eigen values so far i.e.:
      - T = 0
      - T = T + current eigen value

Representing test data points

- PCA solves the optimisation problem

$$W^* = \arg \max_{W^T W = I} W^T X^T X W$$

- whereas Kernel PCA solves :

$$W^* = \arg \max_{W^T W = I} W^T X X^T X X^T W = \arg \max_{W^T W = I} W^T K K W$$

- In PCA the transformed data points are represented using:

$$x_{new} = x_{old} W$$

- Correspondingly, in kernel PCA the transformed data points are represented using:

$$x_{new} = x_{old} X^T W = x_{old} X^T W$$

Representing test data points

- We have:

$$x_{new} = x_{old} X^T W = x_{old} X^T W$$

- In kernel space this becomes:

$$x_{new} = \Phi(x_{old}) \Phi(X)^T W$$

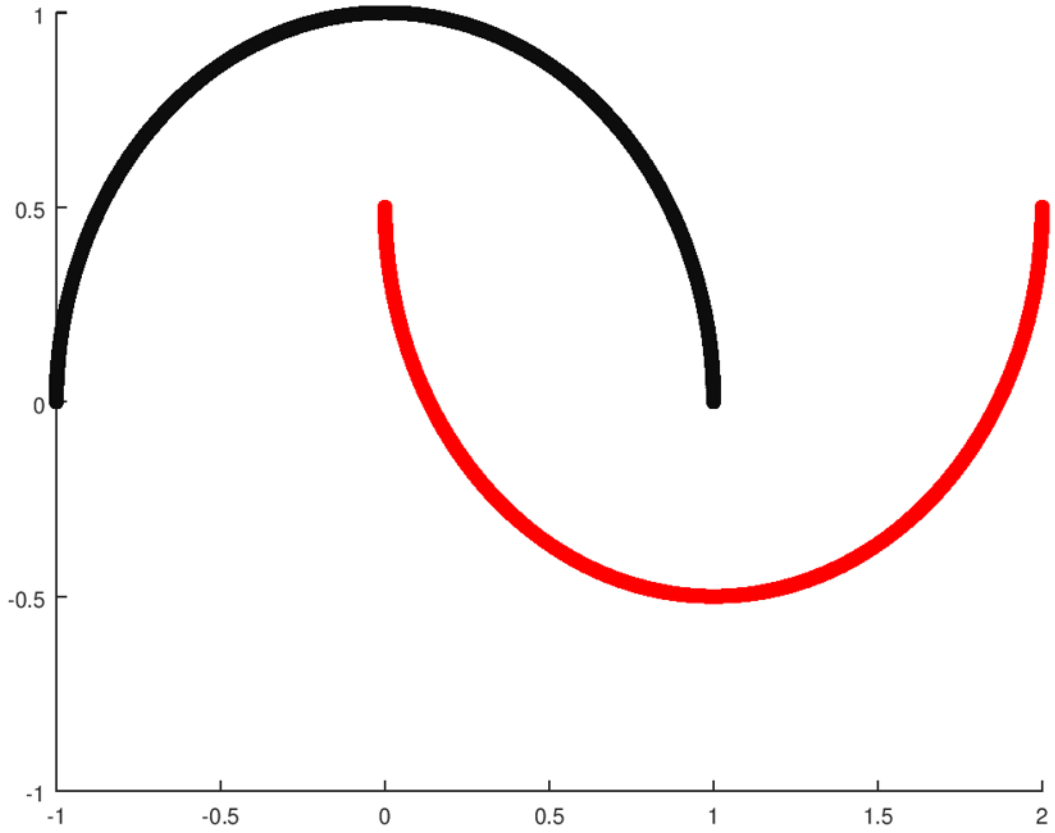
where $\Phi(X)$ represents row wise application $\Phi(x_i)$ to each data row x_i of X

- Translating the above to calls to the kernel, we get:

$$x_{new} = \Phi(x_{old}) \Phi(X)^T W = [K(x_{old}, x_1), \ldots, K(x_{old}, x_N)] W$$

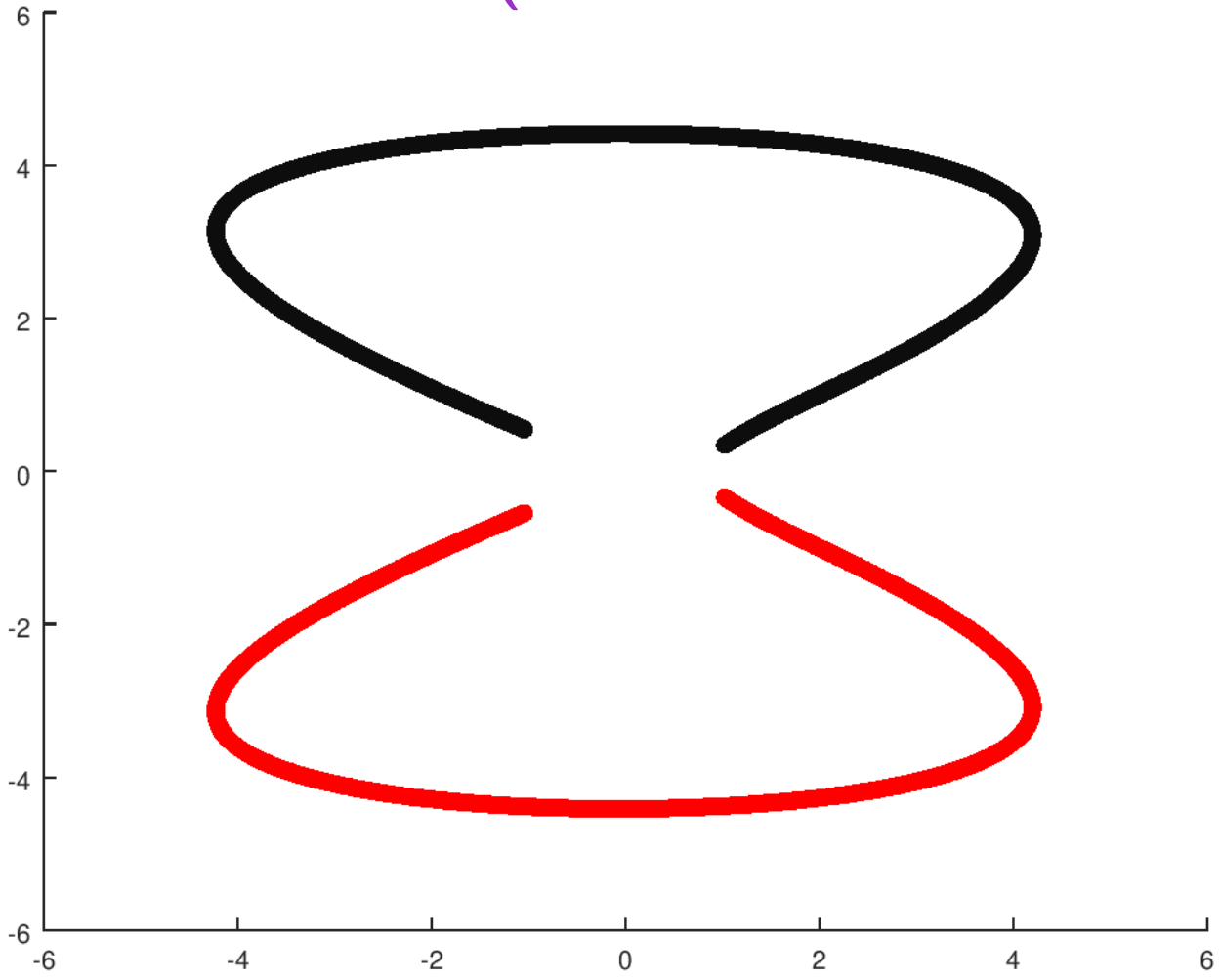
- However, the above does not center the kernel, hence is **incorrect**. A centred version is needed for generating new representation for test data points. (Details omitted here).
- We use the same mechanism as before to centralise
- Hence, new data points are represented as similarities with every data in the training set. This is the feature vector.
- Low dimensional representation is generated by W (by suitably chopping it)

Example (Halfmoon data)



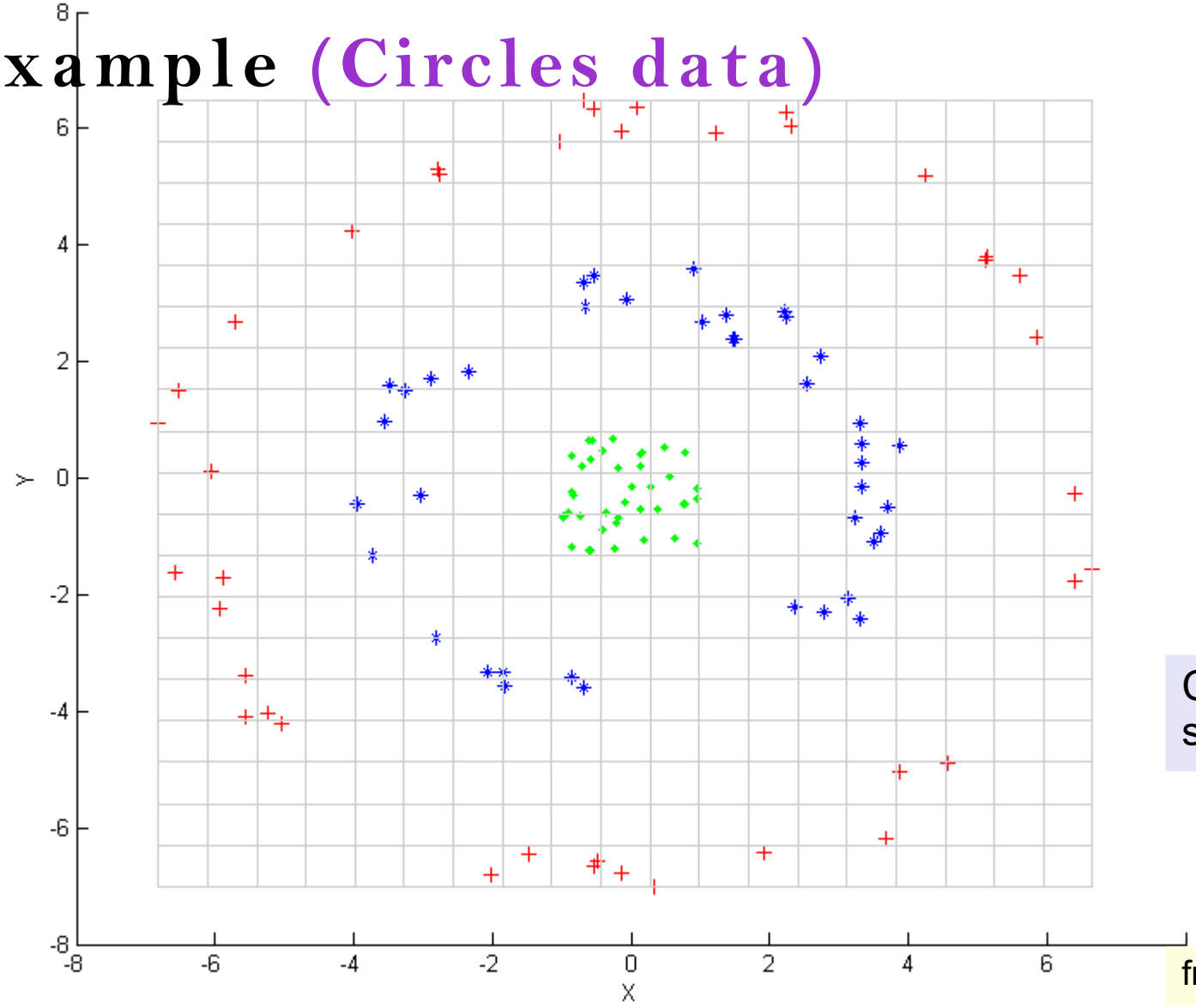
Classes are not linearly separable

Kernel PCA (Halfmoon data – 2 bases)



Shows clear separation between classes with single component using RBF kernel

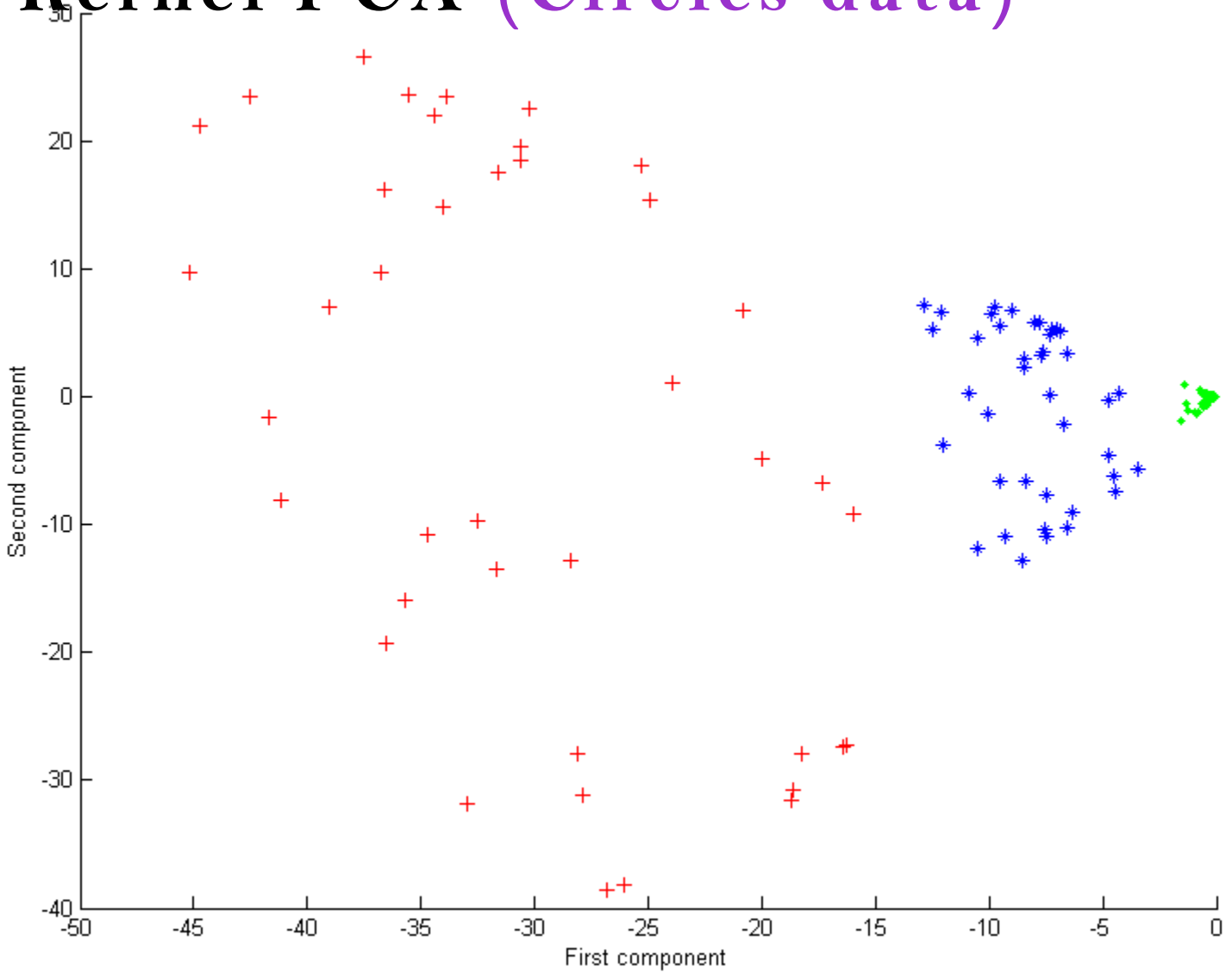
Example (Circles data)



Classes are not linearly separable

from *wikipedia*

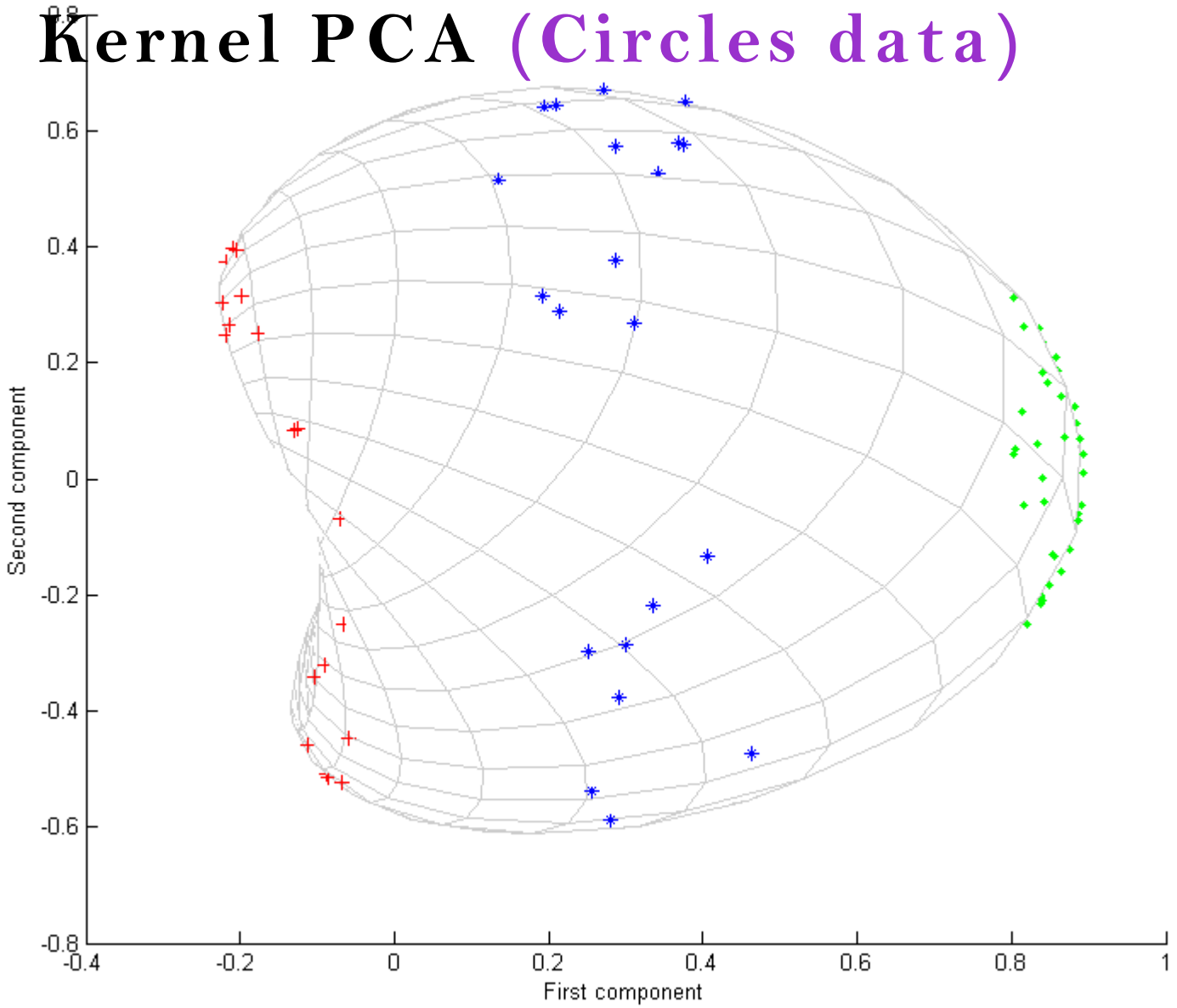
Kernel PCA (Circles data)



Classes are separable with first eigen vector using polynomial kernel $(x.y + 1)^2$

from *wikipedia*

Kernel PCA (Circles data)

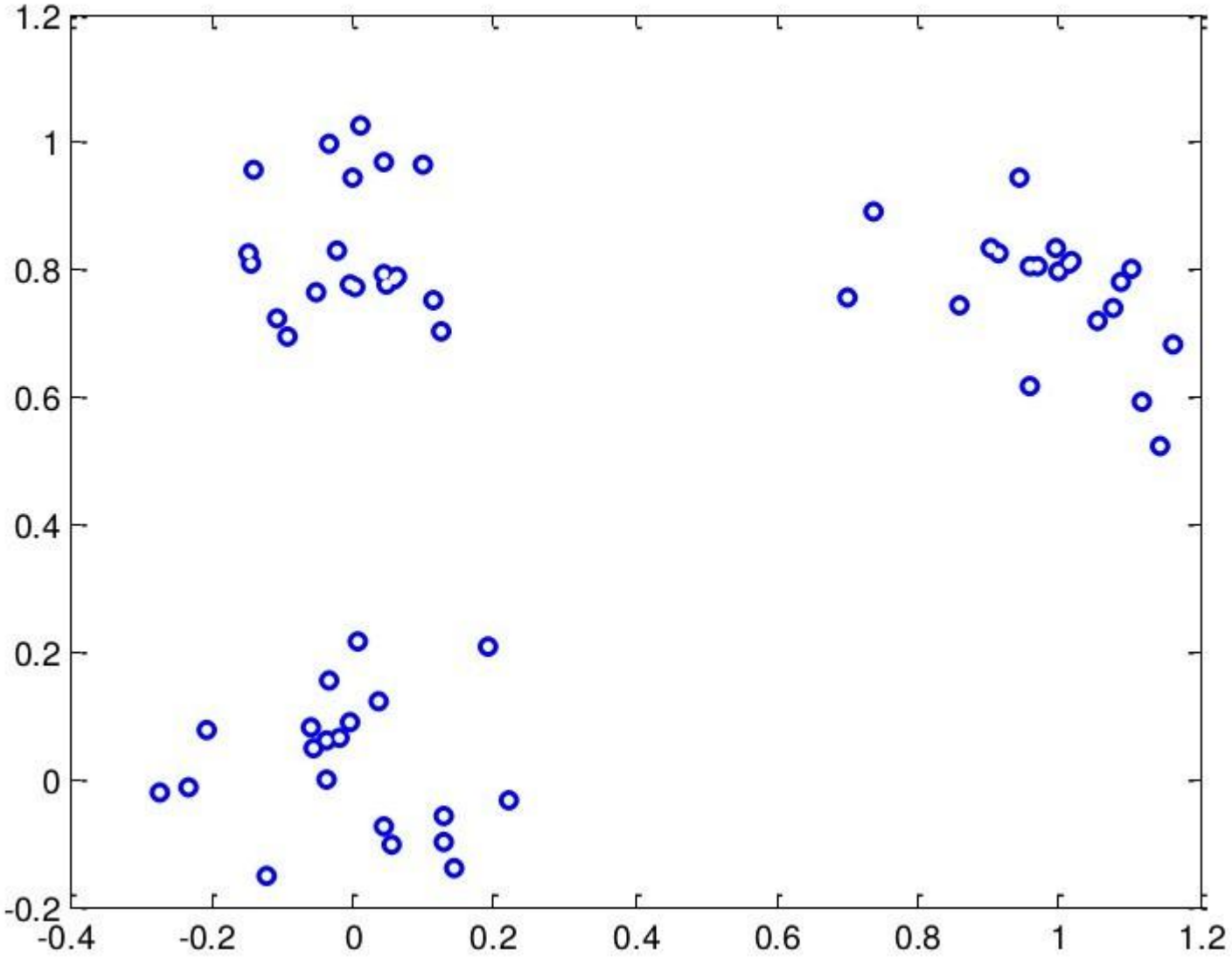


Classes are separable with first eigen vector using RBF kernel

$exp(- \gamma |x - y|^2)$

from *wikipedia*

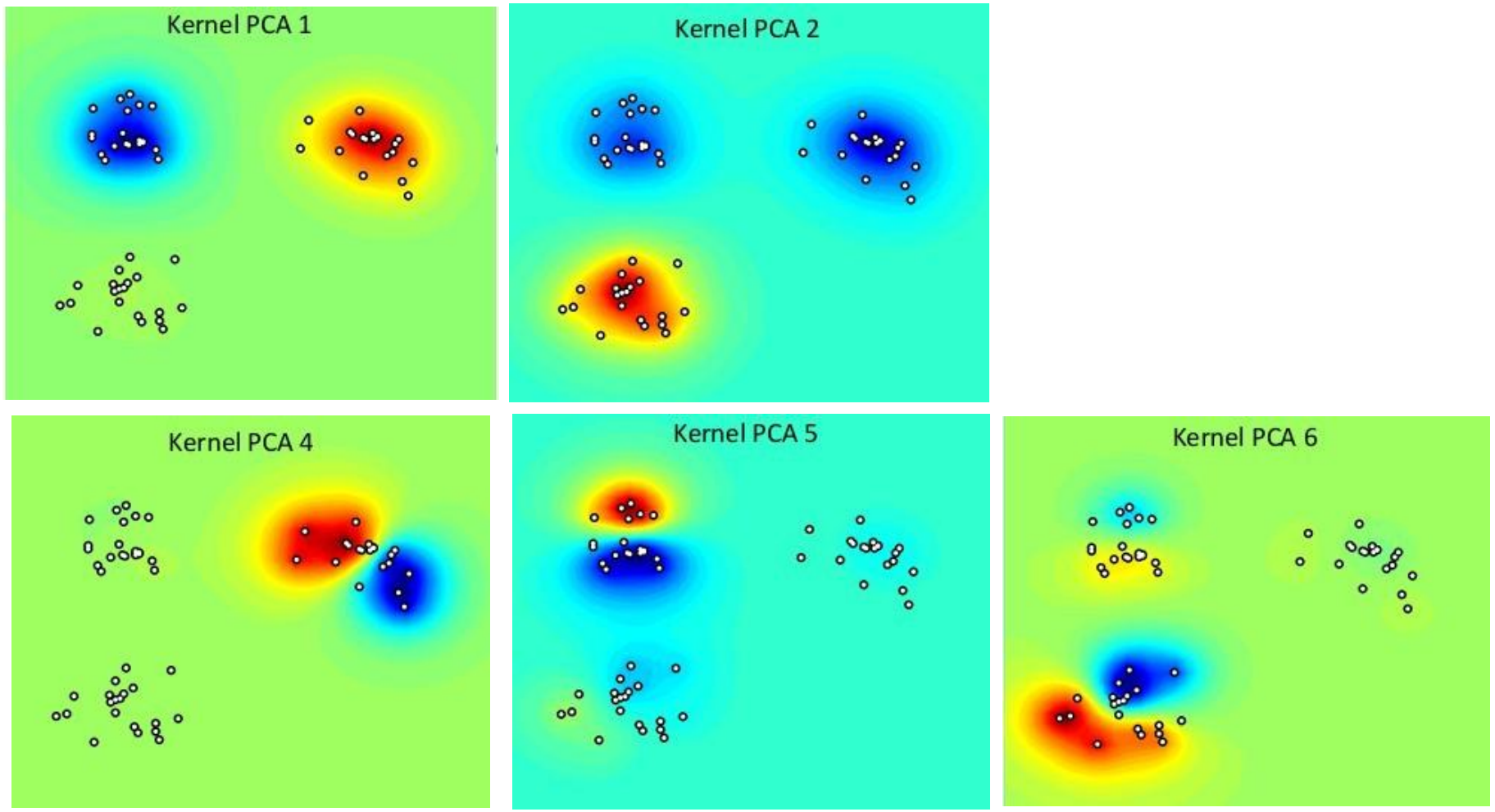
Example (Multiple clusters data)



Dataset containing 3 clusters

from *Ricardo Gutierrez-Osuna CSE@TAMU*

Kernel PCA heatmap (Multiple clusters data)



from *Ricardo Gutierrez-Osuna CSE@TAMU*

Kernel PCA at a glance (what it does)

- The kernel transformation using the kernel trick translates each data point into a ***high dimensional*** (feature) space
- This kernel (feature) space is implicitly represented as similarities between data points
- The similarities are implemented using code as a ***kernel function***
- Kernel functions need to obey certain conditions
- Kernel PCA learns a ***linear combination*** of the features in this high dimensional space
- Each new feature is a linear combination of the similarities with existing data points
- When a reduced dimensionality is chosen then only the strongest features are selected (sorted according to its eigen value)

Summary

- Kernel functions can be used to tackle problems where the data does not have a flat structure
- The kernel function implies a new high-dimensional space for the data but does not actually calculate it
- There are certain conditions for a function to be a kernel function
- We can calculate everything about the data in the new space using the kernel function
- We can adapt algorithms by kernelising them
- They then depend only on the values of the kernel function

Thank You